AI Coding Agent Trust and Pull Request Outcomes in Open Source

Kunwarbir Singh (V01024960), Gabriel Taves (V00939587), Amrinder Singh (V01024880), Luke Thomas (V00976989)
SENG 404, University of Victoria

Abstract. AI coding agents now submit pull requests to open-source repositories, but maintainer trust cannot be measured by merge rate alone. Using AIDev/AIDev-pop, we analyze 33,596 agent-authored pull requests across review sentiment, review engagement, within-repository outcomes, developer experience, PR complexity, and dataset imbalance. We find that agent rankings are highly metric-dependent: Claude Code receives the most positive general sentiment, OpenAI Codex and Cursor perform better under code-review-specific and formal review signals, Cursor has the strongest within-repository acceptance and merge-speed profile, and Copilot receives the most scrutiny when reviewed. However, controlled models show that developer account age and agent choice alone do not robustly explain merge success once PR complexity, author association, repository context, and high-volume author-repository artifacts are considered. Overall, OSS acceptance of AI-assisted PRs appears context-driven: maintainers respond to reviewability, author standing, repository norms, and the kind of work being attempted.

CCS Concepts: Software and its engineering~Open source model; Software and its engineering~Software evolution; Human-centered computing~Empirical studies.

Keywords: AI coding agents, pull requests, open source software, code review, sentiment analysis, software repository mining.

1. INTRODUCTION

AI coding agents increasingly participate in open-source software (OSS) development by generating changes, opening pull requests (PRs), and entering established review workflows. Unlike local autocomplete systems, agent-authored PRs become public artifacts that maintainers must triage, review, request changes on, approve, or merge.

This project studies how agent-authored PRs are received in real OSS repositories. The original framing, 'which coding agent performs best?', was too narrow because OSS acceptance is not only a code-generation problem. Maintainers also evaluate scope, reviewability, author standing, task type, repository norms, and the discussion that surrounds a PR. We therefore analyze agent outcomes as a multi-dimensional trust and context problem.

Our main research question is: to what extent does the choice of AI coding agent affect PR outcomes, including review sentiment, review engagement, acceptance, review duration, and merge success, across task categories after accounting for PR complexity, repository characteristics, developer experience, and dataset imbalance?

We answer this through five linked sub-questions. The first asks how sentiment toward agents differs across agents and over time. The second asks how reviewers engage with agent-authored PRs through coverage, responsiveness, scrutiny, and approval. The third asks whether agents perform differently within the same repositories, where repository norms are held more constant. The fourth asks whether developer experience and author standing explain PR outcomes after controlling for complexity. The fifth asks how dataset imbalance changes the apparent conclusions.

RQ	Focus	Main evidence
RQ1	Review sentiment	VADER, TextBlob, SentiCR, review states
RQ2	Review engagement	Coverage, scrutiny, TTFR, approval
RQ3	Within-repo outcomes	Fixed effects, Cox models, specialization
RQ4	Developer context	GitHub enrichment, author association
RQ5	Imbalance	Downsampling, repo-normalization, high-volume checks

Table 1: Research-question structure.

2. MOTIVATION AND RELATED WORK

The study is relevant to OSS maintainers who must evaluate AI-assisted contributions, tool builders who want to understand how agents are accepted in practice, and researchers who mine GitHub data to evaluate AI systems. A merge-only analysis risks missing how much human review was needed, whether reviewers expressed concerns, and whether an apparent agent effect is actually a repository or author effect.

AIDev provides the empirical basis for this work by collecting agent-authored GitHub PRs [8]. Recent work on AI teammates and AIDev-based agent studies argues that autonomous coding agents are beginning to act more like project participants than editor-local tools [9, 12, 13]. Prior pull-request research shows that acceptance and latency depend on technical and social factors, including project process, contribution context, discussion quality, and review practice [4, 15, 16].

Our sentiment analysis builds on software-engineering sentiment research. General-purpose sentiment tools can

misclassify technical language such as 'bug', 'fail', or 'dead code'. SentiCR, SentiStrength-SE, and SE-specific replications show that tool choice can affect conclusions [1, 7, 10, 11, 18]. Finally, usability work on code-generation tools shows that developer trust depends on workflow context rather than generated code alone [17].

This literature motivates two design choices in our study. First, we avoid reducing agent quality to a single outcome. Prior PR research treats acceptance, review latency, and review process as related but distinct phenomena, so we examine sentiment, engagement, acceptance, merge speed, and controlled merge success separately. Second, we treat GitHub repository mining as a measurement problem. Agent labels, task labels, human-review coverage, and account metadata are all proxies that require robustness checks before being interpreted as evidence of trust.

Our contribution is therefore not a new benchmark of coding ability. Instead, we provide an empirical view of how agent-authored work moves through OSS review pipelines. This complements benchmark-centered evaluations by asking which kinds of signals maintainers appear to respond to when AI-generated changes arrive as pull requests.

3. DATASET AND METHODOLOGY

We use AIDev-pop, an enriched subset of AIDev containing 33,596 agent-authored PRs from repositories with more than 100 stars. The main tables used were pull_request, pr_reviews, pr_comments, pr_review_comments_v2, pr_task_type, repository, user/all_user, pr_timeline, and commit-detail tables. Agents analyzed include OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code. LOC denotes lines of code changed, OR denotes odds ratio, HR denotes hazard ratio, and JSD denotes Jensen-Shannon divergence.

The dataset is intentionally useful but imbalanced. OpenAI Codex accounts for almost two thirds of the observed PRs, while Claude Code appears in a much smaller sample. This matters because a global agent ranking can otherwise be driven by the projects and accounts where a tool is most commonly deployed rather than by the tool itself.

Agent	PRs	Repos	Accounts
OpenAI Codex	21,799	1,248	1,284
GitHub Copilot	4,970	1,012	1
Devin	4,827	288	1
Cursor	1,541	327	363
Claude Code	459	213	236

Table 2a: Agent distribution in AIDev-pop.

Table	Role in analysis
pull_request	Agent label, repository, timestamps, state, merge status
pr_reviews	Review state, approval, changes requested, review timing
pr_comments / review_comments	Issue-thread and inline review text
pr_task_type	Task category controls and specialization
repository / user	Repository context and developer/account proxies
pr_timeline / commits	Events, commits, churn, files, complexity

Table 2: AIDev/AIDev-pop tables used.

The analysis combines four outcome layers. First, sentiment analysis uses 21,922 human external review/comment text items after removing bots, PR-author self-comments, empty comments, very short comments, code blocks, URLs, and generated markup. We compare VADER, TextBlob, a SentiCR-trained code-review classifier, and a formal review-state proxy. Second, review engagement models whether PRs receive human review, changes requested, fast review response, inline/discussion intensity, and low-friction approval.

Source	Count
Inline review comments	10,830
PR issue-thread comments	8,900
Top-level review bodies	2,192
Total human external items	21,922

Table 3: Filtered sentiment corpus.

Third, within-repository analysis restricts to repositories where at least two agents each meet a minimum PR threshold (k=5 and k=10). This controls for repository-level confounds by comparing agents inside the same projects using centered acceptance, paired Wilcoxon tests, fixed-effects logit, Cox merge-speed models, JSD task specialization, and repo-fixed-effects size models. Fourth, the developer-context analysis queried 2,025 PRs through the GitHub REST API [3], successfully enriched 2,003, and added additions, deletions, changed files, commits, author_association, merged/state, and review-duration information.

For review engagement, we use a two-stage framework. Stage 1 asks whether a PR receives any human review at all. Stage 2 studies reviewed PRs or time-to-review models with right-censoring. This distinction matters because an agent can appear to have low scrutiny simply because most of its PRs are never reviewed by humans.

Construct	Operationalization
Coverage	Any human pr_reviews row
Responsiveness	Time to first human review
Scrutiny	n_changes_requested > 0
Inline intensity	Human inline review comments
Discussion intensity	Human issue-thread comments
Low-friction approval	Approval with no change request

Table 4: Review-engagement constructs.

Developer experience is proxied by GitHub account age at PR creation, with followers as a secondary reputation-like proxy. Controlled models include agent, task category, account age, followers, churn, changed files, commits, author association, PR month, and repository-clustered standard errors. We also identify extreme author-repository pairs with more than 100 observed PRs to reduce service-account and high-volume workflow artifacts.

Layer	Primary outcome	Main controls/checks
Sentiment	Tone and review-state signal	Method comparison; repository normalization
Engagement	Coverage, scrutiny, TTFR, approval	Task, language, complexity, repo clustering
Within repo	Acceptance and merge speed	Repo fixed effects; k=5/k=10 thresholds
Developer context	Review, merge, duration	GitHub enrichment; author association; high-volume pairs

Table 5: Analysis layers and controls.

The methodological choice to combine descriptive and controlled models is deliberate. Descriptive tables reveal how agents appear in the dataset and are useful for understanding maintainer-facing experience. However, raw comparisons are not sufficient because agent use is not randomly assigned. For example, some agents are concentrated in particular repositories, some are associated with service-like accounts, and some tend to generate larger or smaller PRs. Controlled models and within-repository restrictions therefore act as checks on whether a raw difference is likely to be an agent effect or a context effect. This follows the broader caution from GitHub mining work that repository data is valuable but can be biased by project selection, incomplete metadata, and changing platform state [19].

We use logistic models for binary outcomes such as review coverage, changes requested, approval, and merge success. We use time-to-event models, including Weibull AFT and Cox models, for responsiveness and merge speed because open, unreviewed, or unmerged PRs can be right-censored. We use negative binomial models for count outcomes such as inline and discussion intensity because review comments are overdispersed: most PRs receive few comments, while a small number receive many. These choices match the measurement properties of the outcomes rather than forcing all questions into a single model type.

For dataset imbalance, we distinguish three threats. The first is agent imbalance, where Codex is much more common than other agents. The second is repository imbalance, where agents are not evenly distributed across projects. The third is author-repository concentration, where a small number of accounts create many PRs in the same repository. These threats are handled with downsampling, repository-normalized sentiment, within-repository comparisons, cluster-robust standard errors, and exclusion of extreme author-repository pairs in robustness models.

4. RESULTS

4.1 Review Sentiment

Review sentiment is generally neutral-to-positive, but it changes by method and context. Over time, general sentiment becomes more neutral once the dataset reaches its high-volume period. January-April 2025 has a VADER mean of 0.155 and a SentiCR negative share of 22.7%, while May-July 2025 has a VADER mean of 0.095 and a SentiCR negative share of 23.8%. The edge months are sparse, so this is an initial temporal pattern rather than a long-term historical trend.

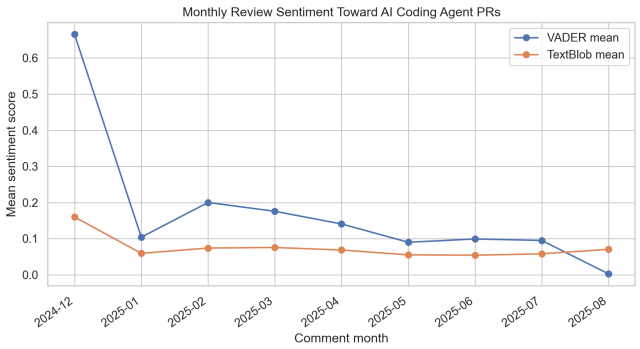

Figure 1: Monthly general sentiment toward agent-authored PR discussions.

Agent	VADER	Pos.	Neg.
Claude Code	0.269	57.8%	13.4%
Cursor	0.147	48.5%	22.0%
Devin	0.141	39.0%	16.7%
OpenAI Codex	0.119	49.8%	23.3%
GitHub Copilot	0.080	36.6%	20.6%

Table 6: General review sentiment by agent.

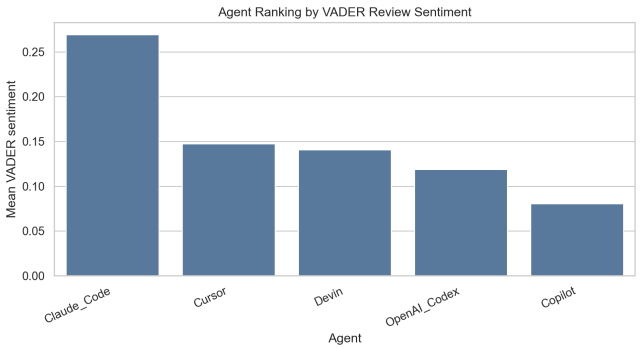

Figure 2: VADER general sentiment by agent.

General sentiment ranks Claude Code highest and GitHub Copilot lowest. However, code-review-specific and formal review signals favor OpenAI Codex and Cursor: OpenAI Codex has a 17.8% SentiCR negative share, a 0.613 review-state score, and a 66.5% approval rate; Cursor has a 20.1% SentiCR negative share, a 0.592 review-state score, and a 63.3% approval rate. Method choice therefore changes the apparent agent

ranking.

Agent	SentiCR neg.	Review-state	Approved
OpenAI Codex	17.8%	0.613	66.5%
Cursor	20.1%	0.592	63.3%
Devin	22.3%	0.334	40.3%
Claude Code	24.7%	0.366	41.1%
GitHub Copilot	24.9%	0.239	36.0%

Table 7: Code-review-specific and formal review sentiment signals.

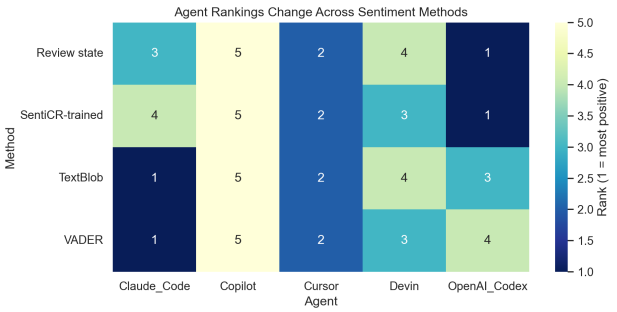

Figure 3: Agent rankings vary across sentiment methods.

Merged PRs have lower SentiCR negative-comment share than unmerged PRs. In a logistic model predicting merge outcome, SentiCR negative share is associated with lower merge odds (coefficient = -0.479, $p < 0.001$), while VADER is not significant. This suggests that code-review-specific negativity is more informative for PR outcome than general affect.

The method disagreement is measurable, not just visual. VADER and TextBlob have a moderate Spearman correlation of 0.441, but their correlations with the SentiCR-trained classifier are near zero (0.075 and 0.063). Kruskal-Wallis tests show significant agent-level differences for both VADER ($H=178.657$, $p<0.001$) and TextBlob ($H=132.305$, $p<0.001$). Thus, agent sentiment differs, but the meaning of 'positive' depends on whether the method captures general tone, code-review negativity, or formal approval.

4.2 Review Engagement

Only 6,004 of the 33,596 PRs (17.9%) receive at least one human review. Review coverage varies substantially by agent: Codex receives 5.4% coverage, Claude Code 24.6%, Cursor 25.5%, Devin 36.9%, and Copilot 51.2%. After controls, Copilot, Devin, and Cursor remain more likely than Codex to receive human review.

Agent contrast	Review-coverage OR	95% CI
Copilot vs Codex	5.99	[3.81, 9.43]
Devin vs Codex	3.63	[1.94, 6.77]
Cursor vs Codex	2.25	[1.19, 4.25]

Table 8a: Controlled human-review coverage contrasts.

Claude Code's coverage contrast is borderline rather than absent (OR 1.79, $p=.054$), which matters because its sample is small. Coverage also varies by task: chore and CI/build PRs attract disproportionately low coverage, while revert PRs are reviewed faster. Copilot and Cursor are also more human-assisted than fully autonomous, so their higher coverage may partly reflect active collaborative workflows rather than reviewers reacting only to generated code.

Agent	Coverage	Scrutiny	TTFR
OpenAI Codex	5.4%	4.9%	2.3h
Claude Code	24.6%	9.7%	12.6h
Cursor	25.5%	5.6%	3.6h
Devin	36.9%	12.5%	1.0h
GitHub Copilot	51.2%	23.8%	1.4h

Table 8: Review engagement summary by agent.

Conditional on review, Copilot receives the most scrutiny: 23.8% of reviewed Copilot PRs receive changes requested, compared with 4.9% for Codex. After controls, Copilot PRs are 3.40 times more likely than Codex PRs to receive CHANGES_REQUESTED, and Devin PRs are 1.81 times more likely. Devin receives review fastest, while Claude Code is slowest, partly due to a smaller reviewed sample. Cursor has the highest raw low-friction approval rate (77.9%), but no agent coefficient is significant in the controlled low-friction approval model.

Task type and complexity are not background noise in this model. Chore, refactor, and fix PRs receive fewer change requests than feature PRs, while each doubling of commit count roughly doubles the odds of CHANGES_REQUESTED (OR 2.04, $p<.001$). A significant agent-by-repository-stars interaction further shows that agent effects are agent-in-context effects, not stable intrinsic properties of a tool across all projects.

Agent	Low-friction approval
Cursor	77.9%
Devin	69.6%
OpenAI Codex	68.5%
Claude Code	62.8%
GitHub Copilot	57.1%

Table 9: Raw low-friction approval among reviewed PRs.

This result is important because it separates review attention from review approval. Copilot is reviewed more often, but its reviewed PRs are also more likely to receive change requests. Cursor is reviewed less often than Copilot but has a more favorable raw approval profile. The controlled model suggests that some of the raw approval gap is driven by repository and task context, reinforcing the need for contextual rather than purely agent-level conclusions.

The engagement findings also show a selection problem. If an agent receives little human review, then low change-request rates do not necessarily mean low reviewer concern; they may simply mean reviewers did not engage. Conversely, high review coverage can expose more problems and therefore increase observed

scrutiny. Intensity is also weak as an independent signal because medians are zero for all agents except Copilot discussion comments. For this reason, the paper treats coverage, scrutiny, responsiveness, intensity, and approval as separate but related review signals rather than combining them into one score.

4.3 Within-Repository Outcomes and Specialization

The within-repository panel contains 37 qualifying repositories and 2,481 PRs at k=5, and 17 repositories with 1,456 PRs at k=10. This design compares agents within the same repositories, reducing the risk that global rankings merely reflect different repository cultures or maintainer strictness.

Measure	k=5	k=10
Qualifying repos	37	17
PRs in panel	2,481	1,456
Codex repos	30	13
Cursor repos	21	10
Devin repos	17	11
Copilot repos	10	3
Claude repos	4	0

Table 10: Within-repository panel feasibility.

Agent	OR k=5	OR k=10	HR k=5	HR k=10
OpenAI Codex	ref.	ref.	ref.	ref.
Cursor	2.01	3.23	1.45***	1.56
Devin	0.78	0.94	0.78**	0.77
GitHub Copilot	0.25	0.37	0.94	0.70
Claude Code	n/a	n/a	0.97	n/a

Table 11: Within-repository acceptance and merge speed. *** p<0.001; ** p<0.01.

Cursor has the strongest within-repo acceptance and fastest merge profile: adjusted OR 2.01 at k=5 and 3.23 at k=10, with merge-speed HR 1.45 at k=5 and 1.56 at k=10. GitHub Copilot has the lowest acceptance profile, while Devin merges slower than Codex. Task specialization is modest but interpretable: Devin is feature-skewed, Copilot leans toward fixes and CI/build, Cursor does more docs/fixes, Codex is the most balanced, and Claude Code skews toward features and refactors on limited evidence.

The robustness checks from the within-repo section support this interpretation. Paired Wilcoxon tests find Codex above Copilot at k=5 (p=.031) and Cursor above Codex at k=10 (p=.016). Raw-to-adjusted logit shifts are small and never reverse signs: Cursor moves from OR 1.92 to 2.01 at k=5 and from 2.94 to 3.23 at k=10, while Copilot stays low. Thus the acceptance gap is not explained away by task mix or PR size.

Size specialization is clearer. Median raw LOC at k=5 is Codex 57, Copilot 74, Devin 150, Cursor 162, and Claude Code 620. This implies that Codex is usually a small-change agent, Cursor and Devin handle larger changes, and Claude Code produces the largest PRs in a thin sample. Larger PRs merge more slowly (Cox HR 0.87 per log LOC), but size does not explain acceptance: Cursor and Devin both produce larger changes, yet Cursor merges faster and Devin slower.

Agent	JSD	Med. LOC	Interpretation
Claude Code	0.381	620	Largest PRs; feature/refactor skew; thin sample
Cursor	0.286	162	Larger PRs; docs/fix mix; strong outcomes
GitHub Copilot	0.264	74	Fix/CI leaning; low acceptance profile
OpenAI Codex	0.250	57	Smallest and most balanced PR profile
Devin	0.232	150	Feature-skewed; slower merges

Table 12: Task specialization summary.

4.4 Developer Experience, PR Complexity, and Repository Context

The GitHub-enriched model shows that raw agent comparisons are confounded by author standing and PR size. Extreme author-repository concentration is especially large for OpenAI Codex (15,019 of 21,799 PRs, 68.9%) and Devin (2,023 of 4,827 PRs, 41.9%). These high-volume pairs can make naive global comparisons look like agent effects when they are partly workflow or service-account effects.

Agent	Pairs	Extreme PRs	Total PRs	Share
Claude Code	2	3	458	0.7%
Cursor	3	112	1,541	7.3%
Devin	10	2,023	4,827	41.9%
OpenAI Codex	17	15,019	21,799	68.9%

Table 13: Extreme author-repository pair concentration.

Agent	PRs	Churn	Files	Commits	Owner/member
Claude Code	117	561	7	2	47.9%
Cursor	280	120	3	2	60.0%
OpenAI Codex	1,606	47	2	1	74.9%

Table 14: GitHub-enriched PR complexity sample.

Task categories also differ. These categories are lightweight keyword-based labels over PR titles and limited body text, so they are descriptive strata and controls rather than gold-label annotations. Documentation and test PRs have high merge rates (85.0% and 80.4%), while performance and security PRs have lower merge rates (54.6% and 64.1%). Security PRs receive the highest human-review and discussion rates, showing that review attention is task-dependent rather than agent-only.

Task	PRs	Merge	Review	Appr.	Discuss.
Feature	9,988	75.4%	11.4%	7.9%	17.4%
Bugfix	5,020	70.6%	15.3%	11.4%	25.7%
Docs	3,549	85.0%	12.9%	10.9%	16.5%
Test	3,110	80.4%	6.3%	4.7%	9.9%
Other	2,264	80.2%	8.8%	6.8%	13.0%
CI/build	1,892	77.5%	12.9%	10.4%	17.8%
Dependency	1,411	76.0%	15.5%	12.0%	21.6%
Refactor	927	71.0%	15.7%	12.1%	21.6%
Security	259	64.1%	25.5%	21.2%	37.5%
Performance	205	54.6%	14.6%	10.7%	24.9%

Table 15: Task-category outcome differences.

Effect	Outcome	OR	95% CI	p
Acct. age	Merge	1.13	[0.92,1.40]	.248
Churn	Merge	0.64	[0.53,0.77]	<.001
Commits	Merge	1.29	[1.08,1.55]	.006
OWNER	Merge	2.45	[1.62,3.71]	<.001
NONE	Merge	0.05	[0.01,0.17]	<.001
Claude	Merge	1.87	[0.85,4.16]	.122
Cursor	Merge	1.07	[0.64,1.78]	.788

Table 16: Controlled GitHub-enriched merge model highlights.

Effect	Review-duration ratio	95% CI	p
Acct. age	0.98	[0.76,1.27]	.901
Churn	1.34	[0.96,1.86]	.082
Commits	1.56	[1.19,2.04]	.001
NONE	7.83	[2.55,24.09]	<.001
Claude	0.75	[0.34,1.64]	.473
Cursor	0.72	[0.43,1.23]	.230

Table 16a: Controlled review-duration model highlights.

Developer experience measured by GitHub account age is not a robust independent predictor of human review, approval, discussion, merge success, or review duration. Instead, churn, commit count, and author_association matter more. Owner-like authors have much higher merge odds, while authors with no repository association have much lower merge odds and longer time to first review. Agent effects for Claude Code and Cursor are not significant against OpenAI Codex in the controlled merge model.

This does not mean developer expertise is irrelevant. It means that account age is too coarse to explain outcomes once more direct trust and complexity signals are present. In OSS review, a maintainer's decision may depend more on whether the author is an owner, member, collaborator, or external contributor than on how old the GitHub account is. It may also depend on contribution history within the specific repository, which is only partially captured by prior observed author-repository PR count and merge rate.

Review duration reinforces this interpretation. Authors with no GitHub association to the repository wait much longer for first review, while agent identity is not a robust predictor of review duration in the enriched model. This suggests that maintainers may triage agent-assisted PRs through ordinary social and maintenance signals rather than treating the agent label as the primary trust cue.

4.5 Dataset Imbalance and Robustness

Dataset imbalance affects what can be concluded. Sentiment rankings are stable under equal-agent downsampling but can change under repository normalization. Within-repository acceptance and merge-speed conclusions are stronger because they compare agents under the same repository context. The developer-context analysis further shows that high-volume author-repo pairs and author association explain important variation that raw agent comparisons miss.

Agent	PR share	Extreme	Raw merge	Non-ext.	Change
Codex	64.9%	68.9%	85.8%	77.5%	-8.3pp
Copilot	14.8%	7.4%	55.0%	55.0%	0.0pp
Devin	14.4%	41.9%	55.5%	53.2%	-2.3pp
Cursor	4.6%	7.3%	74.6%	73.3%	-1.3pp
Claude	1.4%	0.7%	71.3%	71.1%	-0.2pp

Table 17a: Dataset imbalance and high-volume pair sensitivity.

Area	Raw result	After controls
Sentiment	Claude/Cursor strong	Method and repo sensitive
Engagement	Codex low coverage	Partly context dependent
Acceptance	Cursor strong	Stable within repo
Merge speed	Cursor fast, Devin slow	Stable Cox models
Experience	Raw differences visible	Account age not robust
Author context	Agent effects large	Association explains variation

Table 17: Robustness interpretation by outcome layer.

5. DISCUSSION

The central lesson is that agent trust is multi-layered. Claude Code appears strongest under general sentiment, OpenAI Codex and Cursor under code-review-specific/formal sentiment, Cursor under within-repository acceptance and merge speed, Devin under review responsiveness, and Copilot under review coverage but also scrutiny. These are not contradictions; they show that maintainers evaluate different aspects of agent-authored PRs.

The results also connect to course concepts in software repository mining. First, sampling matters: Codex overrepresentation and high-volume author-repo pairs can dominate raw results. Second, operationalization matters: VADER, TextBlob, SentiCR, and review-state proxies measure different constructs. Third, mining GitHub exposes social and technical confounds: author standing, PR size, repository culture, and task type can look like agent effects unless explicitly modeled.

A practical implication for maintainers is that agent-authored PRs should be evaluated through

ordinary maintainability signals: small scope, clear task fit, trusted author context, and reviewable iteration history. A practical implication for tool builders is that better agent output is not only better code generation; it is also producing PRs that fit repository norms and minimize review friction.

Several findings are suggestive but should not be overclaimed. Claude Code's high general sentiment may partly reflect its small and different sample. Copilot's high scrutiny could mean lower quality, but it could also reflect where and how Copilot PRs are used, especially because Copilot and Cursor often involve human-in-the-loop workflows. Codex's low review coverage could reflect automated low-risk workflows rather than maintainer neglect. These ambiguities are exactly why we treat review sentiment, engagement, within-repo outcomes, and controlled merge success as complementary signals.

The results also suggest that future agent evaluations should report multiple normalizations. A global agent-level table is useful for orientation, but it should be paired with within-repository restrictions, task controls, author controls, and sensitivity to high-volume accounts. Without those checks, the analysis risks measuring the deployment pattern of a tool rather than the reviewability of its contributions.

6. IMPLICATIONS FOR PRACTICE AND RESEARCH

For OSS maintainers, the findings suggest that agent-authored PRs should not be accepted or rejected based only on the agent label. The same agent can look strong under one metric and weak under another. Maintainers may benefit from triage rules that emphasize PR size, task type, author association, and reviewability. Small, well-scoped PRs from trusted project participants appear more likely to move smoothly through review than large or weakly contextualized contributions.

For AI coding-agent builders, the results indicate that review success is partly a product-design problem. Agents should not merely generate code; they should produce changes that fit repository norms, make task intent clear, keep PRs appropriately scoped, and support reviewer understanding. An agent that reduces maintainer review effort may be more valuable than one that only maximizes code volume.

For researchers, the study shows that evaluating AI agents in OSS requires careful operationalization. A global merge-rate leaderboard is easy to compute but can be misleading. A stronger empirical design should combine raw descriptive results, method triangulation, repository-aware models, task controls, and explicit treatment of high-volume accounts and dataset imbalance.

RQ	Answer in one sentence
RQ1	Sentiment is neutral-to-positive but method-sensitive and not steadily improving.
RQ2	Review engagement differs: Codex has low coverage, Copilot high scrutiny, Devin fast review.
RQ3	Within repositories, Cursor has the strongest acceptance and fastest merge profile.
RQ4	Account age is not robust; author association and PR complexity matter more.
RQ5	Dataset imbalance changes naive conclusions, especially for Codex and high-volume pairs.

Table 18: Summary answers to research questions.

These implications are relevant beyond the specific agents in AIDev. As agents change rapidly, any static ranking will age quickly. The more durable contribution is the evaluation framework: measure multiple review outcomes, control for context, and treat repository mining as a source of both evidence and bias.

7. REPRODUCIBILITY AND ENGINEERING PRACTICE

The project was structured as a repository-mining study with reusable scripts and generated artifacts. The team analyses use scripts, notebooks, output tables, and source notes that load AIDev/AIDev-pop tables, filter human review/comment data, enrich selected PRs through GitHub, model outcomes, and write CSV/TXT outputs. Separate scripts run imbalance checks and triangulate PR-level sentiment with merge outcomes. Figures are generated from those outputs and stored as reproducible image artifacts.

The team also treated AI assistance as part of the engineering process rather than hiding it. A prompt log is included in the supplementary materials, and the report explicitly distinguishes team-owned analysis from AI-assisted drafting and formatting. This matters because the rubric asks for responsible software-engineering practice, not just a polished final PDF.

Artifact	Role
RQ1/scripts/*.py	Builds filtered sentiment corpus, method comparisons, and imbalance checks
RQ2/scripts/analysis_pipeline.py	Runs review coverage, scrutiny, responsiveness, and approval models
RQ3/scripts/*.py	Runs within-repository acceptance, merge-speed, and specialization analyses
RQ4/scripts/*.py	Runs developer-context, GitHub enrichment, PR-complexity, and imbalance analyses
RQ*/results/ and RQ*/figures/	Stores generated CSV/TXT/JSON/Markdown outputs and report figures
Prompts.md and docs/	Record AI-assistance use, replication notes, and rubric checks

Table 19: Reproducibility artifacts.

A full replication would require access to the AIDev dataset, the GitHub enrichment procedure, and the team's repository containing scripts and outputs. Because GitHub state changes over time, future reruns of API enrichment may not reproduce every field exactly. For that reason, the report treats the enriched subset as a documented analysis artifact rather than assuming the API is a stable database snapshot.

8. LIMITATIONS

This study is observational and cannot establish causality. Human-reviewed PRs are not a random sample, so review-engagement results are affected by coverage selection. Sentiment is a proxy for reviewer reception, not a direct measure of code quality or trust. Task categories are approximate: some are AIDev LLM labels, while developer-context controls use keyword-based task categories.

The GitHub-enriched analysis covers a subset rather than the full dataset. GitHub author_association is an imperfect proxy for repository standing, account age is a broad proxy for developer experience, and follower counts may be snapshot-based rather than time-aligned to PR creation. Agent attribution is also imperfect because some agents operate autonomously while others may assist human authors. Finally, Claude Code has thinner support than other agents in several analyses.

There are also statistical limitations.
Repository-clustered standard errors reduce but do not remove all repository-level confounding.
Within-repository comparisons are more credible for repository context, but they reduce sample size and exclude repositories where only one agent appears.
Logistic, Cox, and negative-binomial models depend on feature choices and cannot capture all maintainer-specific decision processes. The results should therefore be interpreted as robust associations under several reasonable measurement strategies, not as causal estimates of an agent's inherent quality.

Finally, there are ethical and practical limits to mining OSS data. GitHub activity is public, but contributors may not expect their interactions to be interpreted as evidence of trust in an AI tool. We therefore report aggregate patterns rather than naming individual developers or repositories in the analysis. The study is about maintainers' observable review process, not about evaluating individual contributors.

9. CONCLUSION AND FUTURE WORK

We find that AI coding agents differ in reviewer response and PR outcomes, but the differences depend heavily on metric choice and context. The strongest conclusion is not a single best-agent ranking. Instead, OSS acceptance of AI-assisted PRs appears to depend on reviewability, task type, repository context, author standing, and dataset imbalance.

Future work should validate task labels manually, study reviewer comment content beyond sentiment, model repository random effects at larger scale, and connect agent-authored PRs to post-merge quality signals such as regressions, reverts, and follow-up fixes. A richer developer-experience measure based on prior accepted contributions, project tenure, and domain expertise would also improve on account age.

A useful next step would be a mixed-method study that samples review threads from high-disagreement cases: for example, PRs with positive general tone but code-review-specific negativity, or PRs that receive quick review but fail to merge. Such qualitative analysis could distinguish politeness, trust, technical concern, and maintainer workload in ways that automated sentiment labels cannot fully capture.

10. REFERENCES

[1] T. Ahmed, A. Bosu, A. Iqbal, and S. Rahimi. 2017. SentiCR: A customized sentiment analysis tool for code review interactions. In Proceedings of ASE.
[2] F. Calefato, F. Lanubile, F. Maiorano, and N. Novielli. 2017. Sentiment polarity detection for software development. arXiv:1709.02984.
[3] GitHub Docs. 2026. REST API endpoints for pull requests. https://docs.github.com/en/rest/pulls/pulls
[4] G. Gousios, M. Pinzger, and A. van Deursen. 2014. An exploratory study of the pull-based software development model. In ICSE.
[5] C. J. Hutto and E. Gilbert. 2014. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In ICWSM.
[6] Hugging Face. 2026. hao-li/AIDev dataset card. https://huggingface.co/datasets/hao-li/AIDev
[7] M. R. Islam and M. F. Zibran. 2018. SentiStrength-SE: Exploiting domain specificity for improved sentiment analysis in software engineering text. Journal of Systems and Software 145.
[8] H. Li, H. Zhang, and A. E. Hassan. 2026. AIDev: Studying AI coding agents on GitHub. arXiv:2602.09185.
[9] H. Li, H. Zhang, and A. E. Hassan. 2025. The rise of AI teammates in software engineering (SE) 3.0. arXiv:2507.15003.
[10] S. Loria. n.d. TextBlob: Simplified text processing. https://textblob.readthedocs.io/
[11] N. Novielli, F. Calefato, F. Lanubile, and A. Serebrenik. 2021. Assessment of off-the-shelf SE-specific sentiment analysis tools: An extended replication study. Empirical Software Engineering 26.
[12] D. Ogenrwot and J. Businge. 2026. How AI coding agents modify code: A large-scale study of GitHub pull requests. arXiv:2601.17581.
[13] S. Rahman, M. F. Rabbi, and M. Zibran. 2026. A task-level evaluation of AI agents in open-source projects. arXiv:2602.02345.
[14] P. C. Rigby and C. Bird. 2013. Convergent contemporary software peer review practices. In ESEC/FSE.
[15] G. Gousios, M. A. Storey, and A. Bacchelli. 2016. Work practices and challenges in pull-based development: The contributor's perspective. In ICSE.
[16] J. Tsay, L. Dabbish, and J. Herbsleb. 2014. Influence of social and technical factors for evaluating contribution in GitHub. In ICSE.
[17] P. Vaithilingam, T. Zhang, and E. L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In CHI Extended Abstracts.
[18] M. Ortu, B. Adams, G. Destefanis, P. Tourani, M. Marchesi, and R. Tonelli. 2015. Are bullies more productive? Empirical study of affectiveness vs. issue fixing time. In MSR.
[19] G. Gousios and D. Spinellis. 2012. GHTorrent: GitHub's data from a firehose. In MSR.

Appendix A. SUPPLEMENTARY MATERIALS AND REPLICATION PACKAGE

The replication package accompanying the course submission includes the final GitHub repository, the AIDev dataset reference, analysis scripts, generated figures, output files, report source, team contribution notes, data-placement notes, and AI-assistance prompt log. External source links are the AIDev dataset card [6] and GitHub Pull Requests API documentation [3]. Raw AIDev parquet tables are not committed because of size limits; DATA.md describes where to place them for reruns.

Artifact	Location / description
Project repository	https://github.com/lukethomas27/group6-aidev-research-final
Dataset	https://huggingface.co/datasets/hao-li/AIDev
Root docs	README.md; DATA.md; CONTRIBUTIONS.md; requirements.txt; Prompts.md
Replication docs	docs/replication_package_manifest.md; docs/final_report_rubric_checklist.md
RQ1 sentiment	RQ1/scripts/*.py; RQ1/results/*.csv/*.txt/*.parquet; RQ1/figures/*.png
RQ2 engagement	RQ2/scripts/analysis_pipeline.py; RQ2/results/*.csv/*.png
RQ3 within-repo	RQ3/scripts/*.py and notebook; RQ3/results/results.md; RQ3/results/figures/*.png
RQ4/Q6 context	RQ4/scripts/*.py; RQ4/data/github_pr_metrics.csv; RQ4/results/*.csv/*.json/*.md; RQ4/results/figures/*
Report files	reports/final/Project_Final_Report_ACM.pdf; reports/final/build_acm_final_report.py; reports/final/final_report_submission.tex; reports/final/final_report.md
Proposal	reports/proposal/SENG404_Proposal_Group_6.pdf
Interim artifact	reports/interim/Interim_Report_Group_6.pdf

Table A1: Supplementary material checklist.

Appendix B. TEAM MEMBER CONTRIBUTIONS

Member	Primary contribution
Amrinder Singh	Review-sentiment analysis, sentiment method comparison, imbalance checks, report integration.
Gabriel Taves	Review-engagement framework, coverage/scrutiny/responsiveness/approval models, interpretation.
Luke Thomas	Within-repository acceptance, merge-speed, task specialization, and size-specialization analysis.
Kunwarbir Singh	Developer-experience, PR-complexity, GitHub enrichment, author-association, and controlled outcome models.
Whole team	Research-question framing, dataset interpretation, related-work discussion, final editing, and presentation preparation.

Table B1: Team contribution summary.